

Internet Server Technology

Peter Trinh
ptrinh32@gatech.edu

1 INTERNET SERVER TECHNOLOGY

1.1 Personal Experience

My personal experiences with web server technology include many different projects. The most recent project was an inhouse web application for Michael's Arts and Crafts. This application was known as Design Hub. When I joined the team working on this project, the application was already completely developed; however, maintaining and fixing the application was still a huge priority. I worked exclusively on the web client while my partner worked on the web server together with me. Design Hub was designed to be an in-house web application that can only be accessed through Michael's single sign-on portal. Michael's used web clustering for their servers to ensure that Design Hub would always be operational. However, that is the extent of my knowledge on the physical servers Michael's used for this application since I worked on the client. The web client was developed using React and NodeJS which was then deployed and managed through the use of Kubernetes.

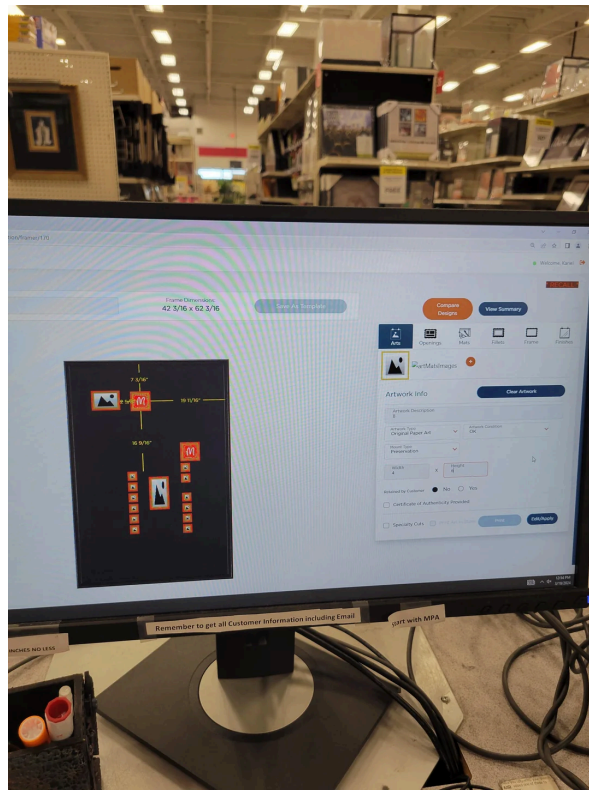


Figure 1—Image of Design Hub. [Source](#)

1.2 Three Techniques

The three techniques used in web server farm systems to improve the scalability of the systems would be clustering the servers, mirroring the servers, and distributing the web servers across multiple servers.

1. Clustering: this technique involves the use of incorporating multiple servers together to host a web service. This basically spreads the burden and work of a singular server so that the web server will be maintained even if one or more servers fails. Scalability of this technique is high since more servers can be added to the cluster as long as there is enough processing power and more CPUs can be added to the servers in the cluster as well
2. Mirroring: this technique involves making copies of the database of the server on a separate disk. Multiple backup sites replicated from the original can be created to ensure that at least one site is still available at all times. The cost of mirroring can be expensive with the replication needed for hardware and operational costs.
3. Distributed web servers: this method uses what's known as load balancing where the traffic on the server is split across multiple designated servers. This is designed to spread out the workload so that no one server is overloaded. This guarantees that the servers have increased traffic speed and incorporates the maintainability of clustering since the servers always have a backup if one goes down.

1.3 Web Browser Cache vs Web Proxy

Web browser cache and web proxy server have the main distinction of where the cached data is stored and how it is used. A web browser cache will store its data on the user's local device, while the web proxy server stores page resources on an intermediate server. There are many benefits with the use of a proxy server, which includes improved network performance, security, and privacy. Web proxy servers allow multiple users to access the same data from the local cache instead of downloading from the original server every time. This improves the speed of the network and reduces traffic. Proxy servers can also enhance security through content filtering. The proxy servers are able to be set to filter out malicious websites and content. In addition, proxy servers improve privacy since the user's actual IP address is masked whenever they visit a site because the proxy server is what interacts with the web server itself. I personally do not have much personal experience working with or using web proxy servers.

2 APPLICATION SERVERS/CLOUD COMPUTING

2.1 Cloud Computing Experience

Technology	User-Perceived Performance	Easy to Use	Functional Design	Non-functional Properties
Amazon Web Services	10	5	8	7
Microsoft Azure	9	6	8	8
Google Drive	10	10	10	9
Microsoft Teams	9	9	9	9
Zoom	8	10	9	10
Slack	6	8	7	7
Heroku	7	7	7	6
Dropbox	8	9	7	7

2.2 MapReduce

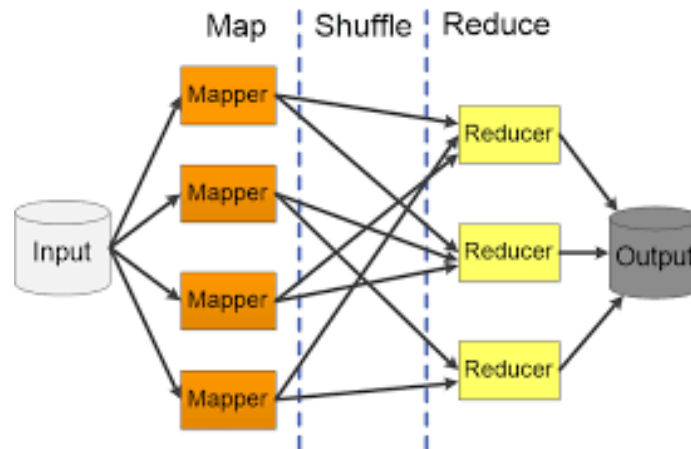


Figure 2—Image of MapReduce. [Source](#)

The MapReduce system is designed to distribute the computing to improve execution performance. The system takes the input data and converts it into a set of data where each individual element is broken down into key value pairs. Then the output from the map is used as an input to combine and create sets of tuples. MapReduce executes the three stages map stage,

shuffle stage, and reduce stage. Within the map stage, the map is used to process the input data which is then used to create smaller tuples of data. The shuffle stage is where the output data from the map stage is redistributed towards the reduce stage. The reduced stage's job is to process the data and produce new sets of output. Each of the three phases assist in distributing the workload to improve the efficiency of working with larger datasets.

2.3 Multi-tier Application Servers

Multi-tier application servers have many differences in functionality compared to web servers. A multi tier application server has the ability to do more than simply displaying static web pages and handle http requests like a web server. Multi-tier application servers offer a much wider range of functionality where they are able to handle complex application logic that allows them to separate between user interface, application processing and data access layers. This gives it dynamic content generation and session management that is unique to these application servers. The main challenge for performance tuning of a multi-tiered application server is due to the separation in the tiers within the application server. Due to the different layers that need to communicate within each other, the challenge of performance tuning is apparent since optimization needs to be done for each tier to ensure efficiency under high work loads. In addition, the communication between each tier will cause latency issues that affect overall time. The complex nature of the multi tier application servers that incorporate multiple tiers for each task within the architecture can also cause performance issues within the server.

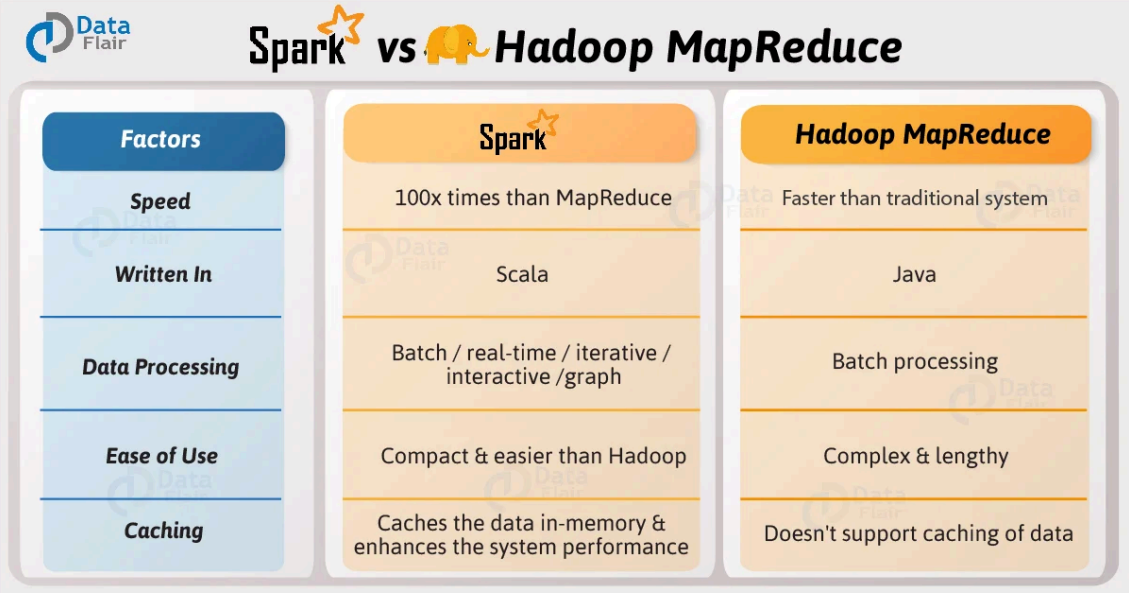
3 CLOUD COMPUTING ECOSYSTEMS

3.1 Three Concepts of Large Scale Data Processing

Clustering, distributed computing, and data partitioning are three different techniques used in large scale data processing. Each concept has its own process that is used in handling massive amounts of data. Clustering involves the grouping of multiple servers together to work on the given data. This clustering of servers allows for parallel execution of work spreading out the resources so that data is processed faster. In addition, this ensures that a failed server can be taken over by working ones. Distributed computing is very similar to clustering; however, the data is divided and sent towards specified servers for handling. All the servers work together to complete the overall computation of the dataset. Distributed computing handles large scale data by combining the processing power of multiple machines together. Data partitioning is the process of subdividing the massive dataset into smaller datasets to be handled. These partitions can then be processed individually through a distributed system. This can help optimize the performance of servers so that a single server is not being overloaded. Unfortunately I have not programmed with Hadoop MapReduce or Spark before.

3.2 Spark vs Hadoop

Personally, I would end up choosing Spark instead of Hadoop for the application development. This is solely for the most efficient and fast performing solution. For a distributed data processing task, I believe speed would be the most important aspect, and Spark performs in memory computation which means it loads data into memory and processes it directly. This increases the overall speed of the processing making it much more efficient compared to Hadoop in the context of processing time. Overall, Spark has the best processing speed between the two. However, even though Spark is faster it still has its negatives. For example, if your goal is for the most cost effective application development then Hadoop far out shines Spark. Spark is extremely memory intensive which makes it inefficient for tasks where memory is a limited commodity. In addition, the cost of in-memory computing requires more resources than the disk implementation of Hadoop. Spark essentially has lower scalability compared to Hadoop because of its high cost requirements.



The image shows a comparison table between Spark and Hadoop MapReduce. The table is titled 'Spark vs Hadoop MapReduce' and is divided into three columns: 'Factors', 'Spark', and 'Hadoop MapReduce'. The 'Factors' column lists five categories: Speed, Written In, Data Processing, Ease of Use, and Caching. The 'Spark' column lists: 100x times than MapReduce, Scala, Batch / real-time / iterative / interactive / graph, Compact & easier than Hadoop, and Caches the data in-memory & enhances the system performance. The 'Hadoop MapReduce' column lists: Faster than traditional system, Java, Batch processing, Complex & lengthy, and Doesn't support caching of data.

	Spark	Hadoop MapReduce
Factors		
Speed	100x times than MapReduce	Faster than traditional system
Written In	Scala	Java
Data Processing	Batch / real-time / iterative / interactive / graph	Batch processing
Ease of Use	Compact & easier than Hadoop	Complex & lengthy
Caching	Caches the data in-memory & enhances the system performance	Doesn't support caching of data

Figure 3—Image of Graph between Spark and Hadoop MapReduce. [Source](#)

3.3 Spark

Spark has a very unique implementation that separates it from Hadoop MapReduce processing, which is its in-memory data processing. The in-memory data processing is where the data from processing is stored within the RAM instead of writing them to the disk. This ultimately improves performance since the in-memory data processing allows for the execution speeds to be much faster than MapReduce.

4 KEY-VALUE SYSTEMS AND DATA WAREHOUSING SYSTEMS

4.1 Key-Value Systems Features

One of the main aspects that separates key-value systems from relational database systems is that there is no schema design. For each key and value within the system the value itself can be defined to be any type of data structure. This gives much more flexibility towards key-value systems since a relational database would require specified schema and structure to the entire system. Another feature is the horizontal scalability of the system itself. Since the data is able to grow without restriction, the system is able to handle large amounts of data efficiently. The data is partitioned also known as sharding across multiple machines. Unlike key-value systems, relational databases have a schema based design. This contrasts the flexibility of the key-value system, which gives relational databases a more rigid design. Both types have their pros and cons for performance, but the key-value system is designed for speed. The system has low latency for read and write operations. On the other hand, relational databases are optimized for more complex data manipulations, but slower overall.

4.2 Big Data vs Conventional Database

Key-value systems have the advantage against relational database systems when storing data because of its simple read and write operations and its unconventional design. In this situation, high volumes of data or unstructured and dynamic data input should always be done with key-value systems. With key-value systems characteristics, large amounts of data can be handled quickly, while unstructured data types can be handled easily as well. Conventional database technology, on the other hand, can be better suited to handle situations where the goal involves handling very structured data and complex querying. Due to the rigidity of conventional databases, structured data can best be handled through these types of databases. In addition, the schema of the conventional databases allows the user to create extremely complex querying that would otherwise be very difficult with key-value systems.

4.3 Technical Challenges

When designing a scalable key-value system, there are multiple issues that can arise. The first issue that is most apparent is the process of data distribution across the system. Due to the unstructured design of the key-value system, the data distribution can be heavily skewed at certain points which can cause specific parts of the system to be overloaded. In addition, another issue that can arise would be key duplication. With any type of key-value system, there is always a possible chance of multiple replica keys causing conflict within the system. This is especially true with a scalable system that is meant to handle large amounts of data. There needs to be some type of mitigation strategy put in place to handle these specific cases. Finally, the last issue that

could come up with is fault tolerance. Whenever large amounts of data are being handled, there are always possible chances of data loss, inconsistencies, and possible node failures. These certain situations can cause a lot of discrepancies throughout the data which needs to be handled properly.

5 REFERENCES

1. *Advantages and limitations of key-value databases | SQL*. (n.d.). Campus.datacamp.com. <https://campus.datacamp.com/courses/nosql-concepts/key-value-databases?ex=4>
2. *Hadoop vs. Spark: What's the Difference? | IBM*. (2024, July 31). Ibm.com. <https://www.ibm.com/think/insights/hadoop-vs-spark>
3. *MapReduce - Understanding With Real-Life Example*. (2020, July 28). GeeksforGeeks. <https://www.geeksforgeeks.org/mapreduce-understanding-with-real-life-example/>